Modern Language Modeling from Scratch

Byte-level BPE and a modern decoder-only Transformer

30 minutes including discussion ▪ Basic Python, tensors, and gradients

The language modeling problem

Given a prefix, predict a distribution over the next token.

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}) \qquad \mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

- The tokenizer defines the discrete symbols x_t.
- The Transformer converts a prefix into vocabulary logits.
- Training learns parameters. Generation repeatedly samples a next token.

Talk route: text encoding, the decoder implementation, then training and inference.

$27\frac{1}{2}$ minutes of slides and $2\frac{1}{2}$ minutes of discussion.

Three different representations of language

Representation	Example and meaning
UTF-8 bytes	`café`: hexadecimal `63 61 66 C3 A9`. A Unicode character can occupy several bytes.
Token IDs	Integers indexing learned byte strings. A token can contain part of a character, a word, or punctuation.
Embedding vectors	$E[x_t] \in \mathbb{R}^d$. Learned continuous features, later updated by context.

BPE learns compression-like segmentation. Neural training learns representations.

Tokenization choices change the modeling problem

Choice	Benefit	Cost or distinction
Whole words	Short sequences for common words	Large vocabulary, unseen-word handling
Characters	Small alphabet	Long sequences, Unicode alphabet choices
Raw bytes	256 base symbols cover UTF-8	Often longer sequences, especially across languages
Byte-level BPE	Byte coverage with frequent strings merged	Vocabulary size trades sequence length for embedding/output cost
Unigram model	Scores alternative segmentations	Different training and segmentation objective

SentencePiece is a tokenizer toolkit supporting BPE and unigram models, rather than a single competing algorithm.

Background: Sennrich et al. (2016), Kudo and Richardson (2018)

BPE training: a small worked example

Illustrative corpus of pre-tokens: `low` occurs 3 times, `lower` 2 times.

`l o w` × 3 `l o w e r` × 2

Step	Pair counts / choice	Resulting representations
Initial	(`l`,`o`) = 5, (`o`,`w`) = 5	Other pairs have count 2
Merge 1	Choose (`l`,`o`)	`lo w`, `lo w e r`
Merge 2	(`lo`,`w`) = 5	`low`, `low e r`

Every merge adds one byte string to the vocabulary and records its rank.

This repository's heap breaks equal counts by *ascending* byte-pair order.

BPE training: the implementation behind the loop

- Split around special tokens, then regex-pre-tokenize ordinary text. Count repeated pre-tokens with multiprocessing.
- Start with 256 byte entries and reserved special tokens.
- Store weighted pair counts and an inverted index from each pair to affected pre-tokens.
- Pop the highest count from a heap. Skip stale entries, merge, and update neighboring pair statistics.

$$C(a,b) = \sum_{w} f(w)\,\#\{\text{adjacent occurrences of } (a,b) \text{ in } w\}$$

Memory depends on distinct pre-tokens, their lengths, indices, and heap entries. Each worker also holds its current text chunk.

Encoding replays learned merge priorities

```
ids = [reverse_vocab[bytes([b])] for b in word.encode("utf-8")]
while an adjacent pair has a learned merge:
    # Lowest rank means earliest merge learned during training.
    i, rank = best_applicable_pair(ids)
    ids[i] = merge_ids[rank]
    del ids[i + 1]
```

- Pseudocode above summarizes `_encode_word`. Encoding uses ranks, not new corpus counts or longest-match search.
- Cache each pre-token's result. Preserve special tokens as dedicated IDs.
- Decode by joining *all bytes first*, then decoding UTF-8.

An individual token may be invalid UTF-8. Arbitrary token lists decode with replacement characters when bytes are invalid.

Token embeddings and the vocabulary projection

$$\underbrace{X}_{B\times T} \longmapsto \underbrace{E[X]}_{B\times T\times d} \longmapsto \underbrace{H}_{B\times T\times d} \longmapsto \underbrace{HW_{\text{out}}^{\mathsf{T}}}_{B\times T\times V}$$

```
# Embedding.forward
return self.embeddings[x]       # E has shape (V, d)

# LinearTransform.forward
return x @ self.weight.T        # weight has shape (out, in)
```

- IDs act as indices. Their numerical distance carries no semantic meaning.
- This implementation learns separate embedding and output matrices: $2Vd$ parameters. Weight tying would share them.

Where this architecture fits

Architecture	Information access	Typical objective / use
RNN language model	Sequential recurrent hidden state	Next-token prediction
Encoder Transformer	Bidirectional self-attention	Masked-token representation learning
Encoder–decoder	Bidirectional source, causal target with cross-attention	Conditional generation, translation
Decoder-only	Causal self-attention over one stream	Next-token prediction and continuation

The original Transformer used an encoder and a decoder. This code implements only a causal decoder stack, with modern block components.

Vaswani et al. (2017), Attention Is All You Need

The complete decoder stack

```
x = token_embeddings(token_ids)       # (B, T, d)
for block in layers:
    x = x + attention(rms_norm_1(x), positions)
    x = x + swiglu(rms_norm_2(x))
x = final_rms_norm(x)
logits = lm_head(x)                   # (B, T, V)
```

- Each sublayer preserves the residual shape (B, T, d).
- Attention mixes information across positions. The feed-forward network operates independently at each position.
- Position enters attention through RoPE on queries and keys.

Pre-normalization and RMSNorm

$$\mathrm{RMSNorm}(x)_i = g_i \frac{x_i}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \epsilon}}$$

- Normalize along the feature axis. The learned scale g starts at one.
- Unlike LayerNorm, RMSNorm does not subtract the mean or add a learned shift in this code.
- Compute the normalization in float32, then restore the input dtype.

$$\text{Pre-norm: } x + F(\mathrm{Norm}(x)) \qquad \text{Post-norm: } \mathrm{Norm}(x + F(x))$$

Pre-norm preserves an explicit identity path through each residual addition.

Zhang and Sennrich (2019)

Causal scaled dot-product attention

$$Q = XW_Q^{\mathsf{T}}, \quad K = XW_K^{\mathsf{T}}, \quad V = XW_V^{\mathsf{T}}$$

$$A = \mathrm{softmax}_{\mathrm{keys}}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_h}} + M\right), \qquad O = AV$$

Queries ask what to retrieve. Keys determine relevance. Values carry the retrieved information. $1/\sqrt{d_h}$ controls score scale as head width grows.

$$M = \begin{pmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{pmatrix}$$

Position t can read positions $\leq t$.

Multi-head attention: following the tensor axes

Use $B = 2$, $T = 4$, $d = 512$, $H = 8$, so $d_h = 64$.

```
Q = x @ weight_q.T                  # (2, 4, 512)
Q = Q.view(B, T, H, d_h).transpose(1, 2)
                                    # (2, 8, 4, 64)
scores = Q @ K.transpose(-2, -1)    # (2, 8, 4, 4)
out = softmax(scores / d_h**0.5, -1) @ V
                                    # (2, 8, 4, 64)
out = out.transpose(1, 2).contiguous().view(B, T, d)
out = output_proj(out)              # (2, 4, 512)
```

The same triangular (T, T) mask broadcasts across batches and heads.

Required: d mod $H = 0$, with even d_h for this RoPE implementation.

RoPE: position as a rotation of queries and keys

For adjacent feature pair i, position m, and $\omega_i = \theta^{-2i/d_h}$:

$$\begin{pmatrix} q'_{2i} \\ q'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\omega_i) & -\sin(m\omega_i) \\ \sin(m\omega_i) & \cos(m\omega_i) \end{pmatrix} \begin{pmatrix} q_{2i} \\ q_{2i+1} \end{pmatrix}$$

$$(R_m q)^\mathsf{T}(R_n k) = q^\mathsf{T} R_{n-m} k$$

- Rotate Q and K, leaving V unchanged. Their dot product depends on relative displacement.
- Precompute sine/cosine tables as nonpersistent buffers. Slice even and odd features, rotate, then interleave.
- Learned absolute embeddings instead add a position vector to the input. RoPE does not guarantee extrapolation beyond training length.

Su et al., RoFormer (2021 preprint)

SwiGLU: a gated feed-forward network

$$\mathrm{SwiGLU}(x) = W_3\,[\mathrm{SiLU}(W_1 x) \odot W_2 x] \qquad \mathrm{SiLU}(z) = z\,\sigma(z)$$

Here x is a column vector. The code applies these transforms to row vectors with transposed weights.

- Two projections expand width d to f. Their elementwise product supplies an input-dependent gate.
- The final projection maps width f back to d.
- Parameters: $3df$, versus $2df_{\mathrm{ordinary}}$ for a two-matrix ReLU/GELU FFN, ignoring biases.

Matching a conventional $f_{\mathrm{ordinary}} = 4d$ gives $f \approx 8d/3$.

Repository detail: the CLI explicitly defaults to $f = 2048$ at $d = 512$. It overrides the module's automatic width selection.

Shazeer (2020), GLU Variants Improve Transformer

Original Transformer and this modern decoder

Component	Original Transformer (2017)	This repository
Structure	Encoder and decoder	Decoder only
Normalization	Post-LayerNorm	Pre-RMSNorm + final RMSNorm
Position	Sinusoidal input embeddings	Rotary queries and keys
Feed-forward	ReLU, two projections	SwiGLU, three projections
Attention	Multi-head + cross-attention	Causal multi-head self-attention
Other choices	Biases and dropout	Bias-free transforms, no dropout

These block choices resemble LLaMA-style models. This is an educational implementation, not a full reproduction of a released model.

Vaswani et al. (2017)

Touvron et al. (2023), LLaMA, Section 2.2

Training examples and stable cross-entropy

```
x = tokens[s     : s + T]        # [A, B, C, D]
y = tokens[s + 1 : s + T + 1]    # [B, C, D, E]
logits = model(x)                # (B, T, V)
loss = cross_entropy(logits.view(B*T, V), y.view(B*T))
```

$$\ell(z, y) = -z_y + m + \log \sum_j e^{z_j - m}, \qquad m = \max_j z_j$$

- The causal mask lets all T positions train in parallel without seeing their future targets.
- Subtract the maximum before exponentiation. Feed raw logits to the loss.
- Perplexity is $e^{\mathcal{L}}$ for natural-log loss. Compare token perplexities only under a matched tokenizer and evaluation setup.

The training loop and its responsibilities

```
lr = warmup_then_cosine(iteration)
x, y = get_batch(train_data, batch_size, context_length, device)
optimizer.zero_grad()
loss = cross_entropy(model(x).view(B*T, V), y.view(B*T))
loss.backward()
gradient_clipping(list(model.parameters()), max_norm)
optimizer.step()
```

- Custom AdamW tracks first and second gradient moments with bias correction and decoupled weight decay.
- Memory-map encoded data. Periodically evaluate held-out loss and save model/optimizer state.
- Fit the tokenizer on training text, then reuse it on validation text. Lower training loss alone does not establish generalization.

Autoregressive generation

$$z = \text{model}(x_{\text{context}})_{\text{last}}, \qquad p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

- Sample one ID, append it to the prefix, and run the model again.
- Temperature $\tau < 1$ sharpens the distribution, $\tau > 1$ flattens it. The code uses argmax when $\tau = 0$.
- Optional top-k keeps the highest logits before sampling.
- This code crops to the last context window and recomputes the entire prefix. It has no KV cache or end-of-text stopping rule.

Training is parallel across positions. Generation is sequential across new tokens.

Scaling variants address different bottlenecks

Technique	What changes	Relation to this code
KV cache	Store past keys and values during decoding	Avoids repeated prefix projections/FFNs
MQA / GQA	Share KV heads across query heads	Reduces KV cache size and bandwidth
FlashAttention	Tile exact attention to reduce memory traffic	Avoids materializing the full score matrix in device memory
Sparse attention	Limit which token pairs interact	Changes connectivity, may reduce attention work

Current dense attention uses $O(BHT^2)$ score memory and $O(BT^2d)$ attention compute. None of these optimizations appears here.

Ainslie et al. (2023), GQA

Dao et al. (2022), FlashAttention

Implementation details that deserve explicit checks

- **BPE ties:** the heap selects lexicographically smaller byte pairs on equal counts. The assignment reference expects the larger pair. Match the chosen specification for reproducibility.
- **Head dimensions:** attention checks only $d_h > 0$. Validate $d \bmod H = 0$ and even d_h before construction.
- **RoPE path:** a block supplies positions automatically. Calling attention directly without positions skips RoPE.
- **Data and checkpoints:** match raw-file dtype to the encoder's output. The script and utility module expose different checkpoint formats.

The output projection has its own weights. A nearby comment claiming it shares the value projection is inaccurate.

The complete model in one explanation

Byte-level BPE learns a reusable vocabulary and merge order.

A causal decoder contextualizes embeddings through RoPE attention and gated feed-forward layers.

Next-token cross-entropy trains the model. Repeated sampling turns its predictions into text.

Discussion: Which tensor changes if we double context length? Which parameter matrices change if we double vocabulary size?

Backup: notation and parameter count

B: batch size, T: sequence length, V: vocabulary size, d: model width, H: head count, $d_h = d/H$, f: FFN width, L: layer count.

$$N = 2Vd + L(4d^2 + 3df + 2d) + d$$

- $2Vd$: separate embedding and language model head.
- $4d^2$: query, key, value, and attention output projections per block.
- $3df$: SwiGLU weights. $2d$: two RMSNorm scales per block.
- Final RMSNorm contributes d. RoPE buffers are not parameters.

For the CLI defaults with illustrative $V = 10{,}000$:

$d = 512$, $f = 2048$, $H = 8$, $L = 6$ gives **35,412,480 parameters**.

This is a calculated configuration, not a reported training result.

Backup: stable masked attention in the source

```
scores = Q @ K.transpose(-2, -1) / Q.shape[-1]**0.5
scores = scores.masked_fill(~mask, float('-inf'))
x_max = torch.max(scores, dim=-1, keepdim=True).values
exp_x = torch.exp(scores - x_max)
weights = exp_x / exp_x.sum(dim=-1, keepdim=True)
return weights @ V
```

- `True` means allowed in this implementation's boolean mask.
- Mask *before* softmax. Multiplying probabilities by a mask afterward loses normalization unless renormalized.
- A fully masked row produces undefined values here. The causal mask includes the diagonal, so ordinary nonempty sequences avoid that case.

Backup: vocabulary and context have different costs

Quantity	Double V, fixed T	Double T, fixed V
Embedding/head params	$2\times$	Unchanged
Output logits memory	$2\times$	$2\times$
Attention score memory	Unchanged	$4\times$
FFN work	Unchanged	$2\times$

Hold other dimensions fixed. In practice, a larger tokenizer vocabulary can shorten a given document's token sequence, coupling these costs.

Backup: checks for a live code walkthrough

- **Tokenizer round trip:** encode/decode valid text with whitespace, accented characters, emoji, and reserved tokens.
- **Causality:** change a suffix and confirm that logits at earlier positions stay unchanged, allowing numerical tolerance.
- **Shapes and gradients:** verify (B, T, V) logits, finite loss, and finite gradients on a small batch.
- **Learning:** overfit a tiny batch, then evaluate separately on held-out data.

These checks establish local behavior. A small model implementing the architecture does not establish large-model capabilities or instruction-following behavior.

References: tokenization and architecture

- Sennrich et al. (2016). Neural Machine Translation of Rare Words with Subword Units.
- Kudo and Richardson (2018). SentencePiece.
- Vaswani et al. (2017). Attention Is All You Need.
- Zhang and Sennrich (2019). Root Mean Square Layer Normalization.
- Shazeer (2020). GLU Variants Improve Transformer.
- Su et al. (2021 preprint). RoFormer: Enhanced Transformer with Rotary Position Embedding.

References: modern decoder and efficient attention

- Touvron et al. (2023). LLaMA: Open and Efficient Foundation Language Models.
- Ainslie et al. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.
- Dao et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.